The National School of Artificial Intelligence
Computer Architecture
Computer Components

Lab 05: Introduction to RARS

Objectives: Introduction to RARS

After completing this lab, you will:

- Learn about the RISC-V assembly language
- Write simple RISC-V programs
- Use system calls for simple input and output

Introduction

RARS, the RISC-V Assembler, Runtime, and Simulator, is a toolchain for the RISC-V ISA. It is a collection of tools that can be used to assemble, run, and simulate RISC-V programs. RARS is a Java application that can be run on any platform that supports Java. It is a simple and easy-to-use toolchain that is perfect for beginners who are learning about the RISC-V ISA. RARS is a great tool for learning about the RISC-V ISA because it provides a simple and intuitive interface that makes writing, assembling, and running RISC-V programs easy. It also provides a built-in simulator that allows you to step through your code and see how it executes on a RISC-V processor.

RARS is a Java application that works under any desktop operating system (Windows, MacOS, Linux). It requires Java 8 or higher to be installed. The latest stable release of RARS can be found here. At the bottom of the page, you will find a .jar file (rars1_6.jar), click on it to download it. After that you just need to run it (it does not need to be installed). Figure 1 shows the UI of RARS.

RARS user interface is composed of three main sections: the editor, the console, and the registers window. The editor is where you write your RISC-V assembly code. The console is where you see the output of your code when you run it. The registers window is where you see the values of the registers in the RISC-V processor when you run your code. RARS also provides a toolbar with buttons for assembling, running, and stepping through your code, as well as buttons for loading and saving files. RARS is a great tool for learning about the RISC-V ISA because it provides a simple and intuitive interface that makes it easy to write, assemble, and run RISC-V programs.

RISC-V Assembly Language Program Template

To write a RISC-V program in RARS, you simply open the editor, click on the "New" button to create a new file, and then start writing your code. Before you can run your code, you need to save it to a file by clicking on the "Save" button. Once you have saved your code, you can assemble it by clicking on the "Assemble" button. If there are no errors in your code, you can run it by clicking on the "Run" button.

A RISC-V assembly language program template is shown below:

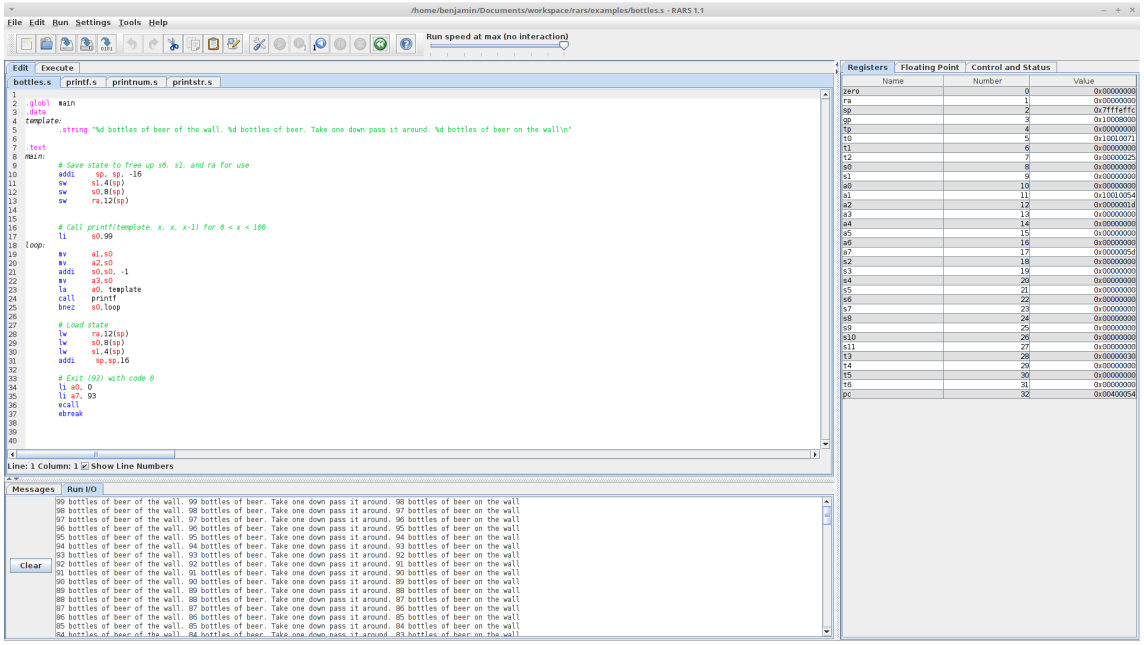

Figure 1: RARS screenshot

```
 .data
# Data section: declare variables and constants
.text
# Code section: write the main function
.globl main
# Declare the main function as global
main:
# Main function entry point
li a0, 10
# Load immediate 10 into register a0
li a1, 20
# Load immediate 20 into register a1
add a0, a0, a1
# Add the values in registers a0 and a1 and store the result in register a0
li a7, 10
# Load immediate 10 into register a7
ecall
# System call to exit
```

Listing 1: RISC-V assembly language program template

There are three types of statements that can be used in assembly language, where each statement appears on a separate line:

Assembler directives

These provide information to the assembler while translating a program. Directives are used to define segments and allocate space for variables in memory. An assembler directive always starts with a dot. A typical RISC-V assembly language program uses the following directives:

- **.data** Defines the data segment of the program, containing the program's variables.
- **.text** Defines the code segment of the program, containing the instructions.
- **.globl** Defines a symbol as global that can be referenced from other files.

Executable Instructions

These generate machine code for the processor to execute at runtime. Instructions tell the processor what to do.

Pseudo-Instructions and Macros

Translated by the assembler into real instructions. These simplify the programmer task.

In addition there are comments. Comments are very important for programmers, but ignored by the assembler. A comment begins with the # symbol and terminates at the end of the line. Comments can appear at the beginning of a line, or after an instruction. They explain the program purpose, when it was written, revised, and by whom. They explain the data and registers used in the program, input, output, the instruction sequence, and algorithms used. For more directives see Appendix A.3.

System Calls

RARS provides a small set of operating system-like services through the system call (ecall) instruction. Register contents are not affected by a system call, except for result registers in some instructions. To issue a system call, follow the following steps:

1. Load the service number (or number) in register a7.
2. Load argument values, if any, in a0, a1, a2 ..., as specified.
3. Issue ecall instruction.
4. Retrieve return values, if any, from result registers as specified.

For a complete list of system calls see Appendix A.2.

Example

```
# Comment giving name of program and description
# sys-call.asm
# Bare-bones outline of RISC-V assembly language program
.globl _start
.data
msg: .asciz "Salem, ensia!\n"
.text
_start:
li a7, 4 # system call code for PrintString
la a0, msg # address of string to print
ecall # Use the system call
li a7, 10
ecall
# End of program, leave a blank line afterwards is preferred
```

Listing 2: System Call Example

Lab Assignment

Write the following programs. Name them P1, P2, P3, and P4. Submit them in a single .rar file.

1. Write a program that asks the user to enter a number N, then prints “Salem” N times.
2. Write a RISC-v program that asks the user to enter his name and then prints “Salem”, followed by the name entered by the user.
3. Write a RISC-v program that executes the statement: s = (a + b) – (c + 101), where a, b, and c are user provided integer inputs, and s is computed and printed as an output.
4. Write a RISC-v program that inputs two integer values. The program should output equal if the two integers are equal. Otherwise, it should output not equal. Use the branch instruction to check for equality.